

The Arrowhead Domain-Decomposition Linear Solver of `dd_solver_simple.hpp`: Mathematics and Design

David Villacis

September 2026

Abstract

`dd_solver_simple.hpp` is a complete, self-contained linear solver for IPOPT built only on Eigen: no HSL (MA57/MA97), no MUMPS. Given a partition of the KKT unknowns into K subdomains plus a shared border, it permutes every Newton system into bordered block-diagonal (“arrowhead”) form, factorizes the subdomain blocks independently by sparse LDL^T , and solves the coupling (Schur-complement) system by preconditioned conjugate gradients — *without ever assembling or factorizing the Schur complement*. The inertia that IPOPT requires is delivered by two applications of the Haynsworth additivity identity: once across the subdomain blocks, and once more across a small “peel” split of the interface, so that the only globally coupled computation left in the algorithm is the inner product reductions of the Krylov iteration. This note records every piece of mathematics the implementation relies on, the failure semantics that make the method safe inside IPOPT’s regularization loop, and the measurements that selected this particular configuration from the larger design space explored in the production solver `dd_solver.hpp`.

Contents

1	The contract with IPOPT	2
2	The arrowhead form	3
2.1	Block elimination	3
2.2	Inertia by Haynsworth additivity	4
3	Subdomain factorization: unpivoted sparse LDL^T	4
4	Forming the local Schur complements	5
4.1	Forward-only formation	5
4.2	Sparse-right-hand-side forward substitution	5
4.3	Row contraction of $Y_k^T D_k^{-1} Y_k$	6
5	The interface operator, matrix-free	7
6	The peel: making the interface fit for CG	7

7	Why the cross points must be peeled	9
7.1	The FETI-DP corner rule	9
7.2	The same conclusion from the optimization side	9
7.3	Measured	9
8	Preconditioned conjugate gradients on S_{ff}	10
8.1	The ASd preconditioner	10
8.2	The iteration	10
8.3	Acceptance, and what happens on rejection	11
8.4	Relation to the iterative strategy of Lueg et al., §3.1	11
9	The predicted inertia	12
9.1	Haynsworth, a second time	12
9.2	Refusing to predict	13
9.3	Why $n_-(S)$ is predictable at all	13
10	Iterative refinement and the outer solve	14
10.1	Life cycle	14
10.2	Constants	14
11	What the measurements decided	15

1 The contract with IPOPT

At every iteration of its interior-point method, IPOPT [1] must solve the primal–dual Newton system. After the barrier terms and its own regularization are folded in, this is a symmetric indefinite *augmented KKT system*

$$A \Delta z = r, \quad A = \begin{bmatrix} H + \Sigma + \delta_w I & J^T \\ J & -\delta_c I \end{bmatrix} \in \mathbb{R}^{n_A \times n_A}, \quad (1)$$

where H is the Lagrangian Hessian, Σ the barrier diagonal (entries scaling like z^2/μ , which is what drives $\|A\|$ to $\sim 10^{18}$ near convergence), J the constraint Jacobian, and $\delta_w, \delta_c \geq 0$ IPOPT’s inertia-correction and dual-regularization shifts, *already applied* by the time the matrix reaches the linear solver. The matrix is handed over in lower-triangle triplet form (i, j, v) , one-based, possibly with duplicate entries that must be summed.

The linear solver must return two things:

1. a solution of $A \Delta z = r$ (possibly for several right-hand sides against one factorization), and
2. the *inertia* of A — specifically $n_-(A)$, the number of negative eigenvalues.

Item 2 is not optional. Writing $\text{In}(A) = (n_+(A), n_-(A), n_0(A))$ for the numbers of positive, negative, and zero eigenvalues, IPOPT’s inertia-correction loop accepts a step only when $n_-(A) = m$ (the number of constraint rows) and $n_0(A) = 0$; otherwise it increases δ_w (and, on singularity, δ_c) and asks for a new factorization. A solver that misreports n_- silently corrupts this loop: the filter line search then fights a curvature defect that does not exist. A solver that cannot report inertia at all forces IPOPT into its inertia-free curvature test [4] — a mode this solver deliberately does *not* use (Section 9 explains what it uses instead, and Section 11 records why).

Three status outcomes cover all failure semantics:

SYMSOLVER_SUCCESS	factorization and solves usable
SYMSOLVER_SINGULAR	factorization unusable; IPOPT raises δ_w, δ_c and retries
SYMSOLVER_WRONG_INERTIA	$n_- \neq m$; IPOPT raises δ_w and retries

Both retry paths are *regularization* paths. A guiding principle of the implementation is therefore: report **SINGULAR** only for conditions that a larger δ_w can actually cure (a broken-down factorization, an undeterminable inertia), and never for conditions it cannot (an iterative solve that failed to converge — see Section 10).

IPOPT owns everything else: the filter line search, the μ -homotopy, the restoration phase. The solver owns only the linear algebra of (1).

2 The arrowhead form

The caller provides an *owner map*: for every KKT index $i \in \{1, \dots, n_A\}$ either a subdomain label $k \in \{1, \dots, K\}$ or the marker “border” for a *complicating* unknown shared by two or more subdomains. (For the image-reconstruction problems this solver was built for, the map comes from a tile or strip partition of the pixel grid; the solver itself is agnostic — it only requires that the map be consistent, in the sense of (3) below.)

Let Π be the permutation that orders all interior unknowns first, grouped by subdomain, and the p border unknowns last. Then

$$\Pi A \Pi^T = \begin{bmatrix} W_1 & & & B_1^T \\ & \ddots & & \vdots \\ & & W_K & B_K^T \\ B_1 & \cdots & B_K & C \end{bmatrix}, \quad (2)$$

with $W_k \in \mathbb{R}^{n_k \times n_k}$ the interior block of subdomain k , $B_k \in \mathbb{R}^{p \times n_k}$ its coupling to the border, and $C \in \mathbb{R}^{p \times p}$ the border–border block. The block-diagonal shape requires that no triplet couple two *different* subdomains directly:

$$\text{owner}(i) = k_1 \wedge \text{owner}(j) = k_2 \wedge k_1 \neq k_2 \implies A_{ij} = 0, \quad (3)$$

which the implementation verifies while routing the triplets and rejects as a “partition leak” otherwise.

Two remarks on (2). First, it is a *pure permutation*: no unknowns are duplicated and no linking constraints are introduced (in contrast to duplicate-and-link reformulations), so C is genuinely nonzero and the B_k carry real Jacobian and Hessian entries. Second, each B_k is extremely sparse: only the border unknowns geometrically adjacent to subdomain k appear in it. Writing $N_k \subseteq \{1, \dots, p\}$ for the set of border positions subdomain k actually touches, with $p_k = |N_k|$, the matrix B_k has nonzero rows only on N_k . We use the same symbol for the selection operator $N_k \in \mathbb{R}^{p_k \times p}$ picking out those positions.

2.1 Block elimination

Assume for the moment every W_k is nonsingular (Section 3 handles failure). Eliminating the interior blocks from (2) produces the *interface* (Schur complement) matrix

$$S = C - \sum_{k=1}^K B_k W_k^{-1} B_k^\top \in \mathbb{R}^{p \times p}, \quad (4)$$

and the three-stage solve for a right-hand side split as $r = (r_1, \dots, r_K, r_y)$:

$$r_S = r_y - \sum_{k=1}^K B_k W_k^{-1} r_k, \quad (5)$$

$$\Delta y = S^{-1} r_S, \quad (6)$$

$$\Delta x_k = W_k^{-1} (r_k - B_k^\top \Delta y), \quad k = 1, \dots, K. \quad (7)$$

Everything indexed by k is independent across k — that is the point of the decomposition. Only (6) is globally coupled, and the design question of this whole solver is how to perform (6) *without a global factorization*.

2.2 Inertia by Haynsworth additivity

Theorem 1 (Haynsworth inertia additivity [2]). *Let $M = \begin{bmatrix} E & F^\top \\ F & G \end{bmatrix}$ be symmetric with E nonsingular. Then*

$$\text{In}(M) = \text{In}(E) + \text{In}(G - FE^{-1}F^\top).$$

Applied to (2) with $E = \text{diag}(W_1, \dots, W_K)$ and Schur complement (4),

$$\text{In}(A) = \sum_{k=1}^K \text{In}(W_k) + \text{In}(S). \quad (8)$$

The first term is *free*: each $\text{In}(W_k)$ is read off the pivot signs of a factorization that must happen anyway, locally and in parallel (Section 3). The second term, $\text{In}(S)$, is the one quantity that would classically force assembling and factorizing S — the single serial step in an otherwise embarrassingly parallel scheme. Section 9 removes it.

Remark 1 (Why pivot signs, not eigenvalues). *On these problems $\|A\|$ reaches $\sim 10^{18}$ through the barrier terms Σ , so the signs of small eigenvalues are rounding noise and any eigenvalue-counting approach is hopeless. The pivot signs of an $LDL^\top$ factorization, by Sylvester’s law of inertia, count $n_\pm$ exactly regardless of scaling: $\text{In}(A) = \text{In}(D)$ whenever $A = LDL^\top$ with L unit triangular. Every inertia statement in this solver ultimately reduces to a pivot-sign or exact-eigendecomposition count of a well-scaled small matrix.*

3 Subdomain factorization: unpivoted sparse $LDL^\top$

Each W_k is factorized by Eigen’s `SimplicialLDLT`:

$$W_k = P_k^{-1} L_k D_k L_k^\top P_k, \quad (9)$$

where P_k is a fill-reducing permutation (computed once per sparsity pattern; the KKT pattern is constant over a whole run, so the symbolic analysis is reused by every Newton step), L_k is *unit* lower triangular, and D_k is diagonal. Eigen applies no scaling, a fact Section 4 depends on. The inertia is $\text{In}(W_k) = \text{In}(D_k)$, counted from the signs of the pivots.

`SimplicialLDLT` does *not* pivot for stability, and this is the one real weakness of the Eigen route. The saving grace is a structural property of (1):

Proposition 1 (Quasidefiniteness [3]). *If $\delta_w > 0$ makes the $(1,1)$ block of (1) positive definite and $\delta_c > 0$, then A is symmetric quasidefinite, and every symmetric permutation of A admits an $LDL^\top$ factorization with nonzero pivots. The same holds for every W_k , which inherits the saddle-point structure.*

IPOPT, however, usually offers a matrix with $\delta_w = \delta_c = 0$ first, and there a zero pivot is possible. The implementation detects breakdown exactly (a zero or non-finite entry of D_k) and reports `SINGULAR` — refusing both the solve and the inertia, since neither is meaningful past a broken pivot. IPOPT responds with $\delta_w, \delta_c > 0$, which is *precisely* the regularization that makes the unpivoted factorization exist. The failure mode is therefore self-correcting; its cost is a few extra factorizations per run relative to a pivoting Bunch–Kaufman code such as MA57, and that is the honest price of dropping HSL. No artificial shift is ever added inside the solver: masking a rank deficiency locally would corrupt the inertia signal (8) that drives IPOPT’s δ_w loop.

4 Forming the local Schur complements

The dense $p_k \times p_k$ matrices

$$S_k = -B_k W_k^{-1} B_k^\top \quad (10)$$

(so that $S = C + \sum_k N_k^\top S_k N_k$ in selection-operator form) are the workhorses of everything downstream: the interface operator, the preconditioner, and the peel all consume them. Note that S_k is *mathematically* dense — entry (a, b) equals $-b_a^\top W_k^{-1} b_b$ with W_k^{-1} full, so generically every pair of border unknowns touching subdomain k interacts — hence “avoid storing it densely” is not an available option; the design question is only how cheaply it can be *formed*. Formed naively, (10) costs p_k full sparse back-solves (forward, diagonal, backward) with dense right-hand sides and dominates the entire factorization. Three exact observations remove most of that cost.

4.1 Forward-only formation

Substituting (9) into (10) and using $P_k^{-1} = P_k^\top$,

$$B_k W_k^{-1} B_k^\top = B_k P_k^{-1} L_k^{-\top} D_k^{-1} L_k^{-1} P_k B_k^\top = Y_k^\top D_k^{-1} Y_k, \quad Y_k = L_k^{-1} P_k B_k^\top \in \mathbb{R}^{n_k \times p_k}. \quad (11)$$

The backward substitution $L_k^{-\top}$ cancels symbolically against the B_k that would multiply it back from the left, so *the backward half of the solve is never performed*, and B_k itself disappears from the product: Y_k already carries it. (This identity requires that the factorization apply no diagonal scaling between the two triangular halves — true for Eigen, not for MA57 under MC64 scaling.)

4.2 Sparse-right-hand-side forward substitution

Each column of $P_k B_k^\top$ carries only a handful of nonzeros (a border unknown couples to a few interior unknowns), and a forward substitution propagates column j of L_k only through the

multiplier x_j : solving $L_k x = b$ column-by-column in ascending j ,

$$x_i \leftarrow x_i - (L_k)_{ij} x_j \quad \text{for all } i > j \text{ with } (L_k)_{ij} \neq 0,$$

a step that contributes *exactly nothing* when $x_j = 0$. The columns of Y_k therefore acquire nonzeros only on the *reach* of the right-hand-side pattern — by Gilbert’s theorem [5], the union of elimination-tree paths from the pattern’s positions to the root. Because the KKT pattern is fixed for the whole run, each column’s reach is *static*: it is computed once per sparsity pattern (walking the etree, which the symbolic analysis already built) and the substitution iterates exactly that list, ascending — a valid order since an etree parent always carries the larger index. Measured on the target problems the reach covers 2–4% of the columns of L_k ; a pattern-blind implementation that instead sweeps all n_k columns testing $x_j = 0$ is algebraically identical, but at $N = 64$ the empty sweep was profiled at more than the arithmetic it guarded, and the same $O(n_k p_k)$ dense-bookkeeping trap (scanning and zeroing the mostly-empty Y_k buffer) applies to the contraction below — so Y_k is held *compressed on its reach* throughout. (Eigen’s own sparse-RHS solve path does none of this; it expands the right-hand side into dense panels.)

4.3 Row contraction of $Y_k^\top D_k^{-1} Y_k$

The remaining cost of (11) is the contraction $Y_k^\top D_k^{-1} Y_k$. As a dense GEMM this costs $2 n_k p_k^2$ flops — almost all of them multiplying the zeros that Section 4.2 just avoided computing (measured: Y_k is $\approx 88\%$ zeros at $N = 32$). Contracting by *rows* instead,

$$S_k = - \sum_{r=1}^{n_k} \frac{1}{(D_k)_{rr}} y_r^\top y_r, \quad y_r = \text{the } r\text{-th row of } Y_k \text{ (sparse)}, \quad (12)$$

costs $\sum_r \text{nnz}(y_r)^2 \leq p_k \text{nnz}(Y_k)$ flops, skips only terms that are exactly zero, and produces an exactly symmetric S_k by construction (the upper triangle is accumulated and mirrored — exact symmetry is relied on by the inertia count of Section 9 and by the preconditioner). The row grouping is read off the same static reach lists by a counting sort, so no dense scan of Y_k ever occurs. Measured whole-run effect of the two formation steps (cameraman, exact Hessian, iteration-for-iteration identical in every cell):

run	dense GEMM	row contraction	+ static reach	total
$N = 32, 3 \times 3$ tiles	4.42 s	4.00 s	3.79 s	1.2×
$N = 64, 2 \times 2$ tiles	42.8 s	20.3 s	16.0 s	2.7×
$N = 64, 4$ strips	25.7 s	10.1 s	8.08 s	3.2×

Remark 2 (Why S_k is nevertheless *kept* dense). *Conjugate gradients only needs the action $p \mapsto S p$, so one could skip forming S_k entirely: either apply the factored form $-Y_k^\top (D_k^{-1} (Y_k p))$ each iteration, or go fully matrix-free and run one W_k back-solve per iteration with no Y_k at all. Measured, both lose. The factored application costs $\approx 4 \text{nnz}(Y_k)$ flops per iteration against $2p_k^2$ for the dense matrix-vector product, and $\text{nnz}(Y_k) > p_k^2$ on most subdomains; at the observed $\sim 10^3$ CG iterations per factorization the saved contraction is repaid with interest. The fully matrix-free application costs on the order of the union reach of $\text{nnz}(L_k)$ per iteration — roughly $10\times$ the dense product — and loses by more. Moreover the ASd preconditioner (Section 8.1) consumes the entries of S_k , not its action, so the dense block must exist regardless. The winning combination is (12) on static reaches: sparse arithmetic to form S_k , dense arithmetic to apply it.*

A runtime referee is kept for all of this: with `DDS_SCHUR.CHECK=1` every S_k is recomputed by the naive two-sided route and compared, per block per factorization (relative error $\sim 10^{-12}$ typical, worst $\sim 10^{-8}$ at the most ill-conditioned iterates).

5 The interface operator, matrix-free

S itself is *never assembled*. Its action is evaluated as the same sum an assembly would have performed,

$$S y = C y + \sum_{k=1}^K N_k^T S_k (N_k y), \quad (13)$$

with every term local to one subdomain. This is the distributed algorithm, not a simulation of it: each rank owns its dense S_k , and the only communication per application is one reduction over the border vector. (The stricter regime in which even the S_k are not kept, and Sy is re-derived through K subdomain back-solves per iteration, is measured in the production solver as `APPLY_MATFREE`; keeping S_k — formed once per Newton step, locally — is the normal choice.)

The diagonal of S , needed by the preconditioner, is likewise assembled without S :

$$\text{diag}(S) = \text{diag}(C) + \sum_{k=1}^K N_k^T \text{diag}(S_k), \quad (14)$$

one all-reduce over a p -vector in a distributed setting — which is exactly why the ASd preconditioner is built around it.

6 The peel: making the interface fit for CG

Conjugate gradients requires a symmetric *positive definite* operator, and S is generally not one. *Three* kinds of border index are responsible:

1. **Dual border indices.** On a $k \times k$ tile partition the driver promotes the cut-corner dual pairs to the border (otherwise the local blocks are structurally rank-deficient). Empirically the promoted duals are *exactly* the negative eigenvalues of S : at every recorded solve, $n_-(S)$ equalled the promoted count. (Section 9.3 explains why.) So on tile partitions S is indefinite *by construction*, and plain CG can never run on it.
2. **The scalar α .** The problem's single global regularization parameter appears in every subdomain, so its row and column of S are dense, wrecking both sparsity and conditioning.
3. **The cross points** — border unknowns touched by three or more subdomains. Unlike (i) these do not make S indefinite; they make it increasingly *ill-conditioned* as the subdomain count grows, which is a subtler defect and the subject of Section 7.

The *peel* removes both before the Krylov solve. Split the border index set into the peeled set P (all three categories above) and the kept set f , and partition

$$\begin{bmatrix} S_{ff} & S_{fP} \\ S_{fP}^T & S_{PP} \end{bmatrix} \begin{bmatrix} \Delta y_f \\ \Delta y_P \end{bmatrix} = \begin{bmatrix} r_f \\ r_P \end{bmatrix}. \quad (15)$$

With

$$Z = S_{\text{ff}}^{-1} S_{\text{fp}} \quad \text{and} \quad T = S_{\text{pp}} - S_{\text{fp}}^{\text{T}} Z \in \mathbb{R}^{|P| \times |P|} \quad (16)$$

(the dense Schur complement of the split), block elimination of (15) gives

$$\Delta y_{\text{P}} = T^{-1} (r_{\text{P}} - S_{\text{fp}}^{\text{T}} S_{\text{ff}}^{-1} r_{\text{f}}), \quad (17)$$

$$\Delta y_{\text{f}} = S_{\text{ff}}^{-1} r_{\text{f}} - Z \Delta y_{\text{P}}. \quad (18)$$

The indefinite and dense directions now live entirely inside T , and S_{ff} — the operator CG actually iterates on — is symmetric positive definite whenever the design premise holds. This is the FETI-DP/BDDC corner treatment [11, 12] in Schur-complement clothing.

The size of P is set by the cross points. On a $k \times k$ tile partition, measured: there are $(k-1)^2$ interior cross points, each of degree exactly three, and $4(k-1)^2$ promoted duals (four per cut-corner cell, since the driver promotes two rank-one dual pairs there). With α ,

$$|P| = 5(k-1)^2 + 1 \quad (\text{measured: } 6, 21, 46 \text{ at } k = 2, 3, 4), \quad (19)$$

against interfaces of $p = 128, 261, 400$. Strip partitions and 1D problems have no cross points and no promoted duals, and there $|P| = 1$. Equation (19) is worth reading carefully: $|P|$ grows *linearly* with the number of subdomains k^2 , so T — which is precisely the coarse problem of this scheme, in the FETI-DP sense — is only cheap at modest k . That growth is the classical scalability limit of a coarse space solved directly, and it is where the multilevel and inexact-coarse-solve literature applies [8, 9]. The implementation therefore carries a guard that declines the cross-point peel outright if it would not leave $|P|$ a small fraction of p : a partition without a small coarse space of this kind is better served by the α and dual peel alone than by moving the whole interface into a dense direct solve.

Building the peel cache. Once per factorization:

1. S_{fp} and S_{pp} are read off by applying (13) to the $|P|$ unit vectors of the peeled positions — no assembly of S is needed;
2. Z is computed one *CG solve per column* of S_{fp} (every system involving S_{ff} in this solver, including these, goes through CG — the honest matrix-free discipline, and also the expensive part). Each column is *warm-started* from its last accepted value: the systems change slowly along the barrier path — across a δ -retry only the regularization moves — and per-column telemetry showed the dominant failure mode was a *cold-start stall* (zero progress from $x_0 = 0$ for the entire stall window, on a column that solved fine one build later), which a nonzero start sidesteps by handing CG a different Krylov space;
3. T is formed densely from (16) and factorized by LU for (17);
4. $\text{In}(T)$ is computed for the inertia prediction (Section 9).

The action of S_{ff} on a vector is obtained for free from (13): embed with zeros on the peeled positions, apply S , and read back the kept positions.

If any column of Z fails to converge, the cache is declared unbuildable and the whole factorization is reported **SINGULAR** (Section 9 explains why refusal is the correct semantics).

7 Why the cross points must be peeled

Categories (i) and (ii) of Section 6 are forced: without them S_{ff} is indefinite or dense and CG cannot run at all. Category (iii) is different — S restricted to the non-cross-point interface is definite, so CG *runs*; it simply converges worse and worse as the subdomain count grows. This section records why that happens and what it costs.

7.1 The FETI-DP corner rule

This is not an analogy with domain decomposition; it is the same construction. Farhat et al. [11] define a corner as, in their Definition D1, “its cross points — that is, the points belonging to more than two subdomains,” and make exactly those unknowns *primal*: globally continuous, eliminated by static condensation into a coarse problem, while the rest of the interface is left to the iteration. Their coarse operator

$$K_{cc}^* = K_{cc} - \sum_s (K_{rc}^s B_c^s)^\top (K_{rr}^s)^{-1} (K_{rc}^s B_c^s)$$

is precisely the T of (16), and their reason for the choice is also ours twice over: making the corners primal is what makes each local block K_{rr}^s nonsingular, which is exactly the rank deficiency this project’s driver cures by promoting cut-corner duals.

What makes the rule sufficient *in two dimensions* is a theorem. Mandel and Tezaur [6] prove that with cross-point constraints alone the condition number of the resulting FETI-DP operator is bounded by $C(1 + \log(H/h))^2$, *independently of the number of subdomains*. The edge and face averages that dominate the BDDC literature are a three-dimensional requirement — in 3D, corners alone lose a factor H/h [7] — and are not needed here. The corresponding statement for the interface *without* a coarse component is the uniform diagnosis of the field: convergence degrades with the subdomain count because only local information is propagated [7, 11].

7.2 The same conclusion from the optimization side

The ASd preconditioner of Section 8.1 is taken from Lueg et al. [13], and that paper reaches the identical conclusion about its own method: “it is well known that so-called one-level preconditioners as described above do not scale well with the number of partitions $n_P \dots$ Improved scalability is obtained by two-level methods, which add a coarse grid correction term.” They give the two-level form $M_{2AS}(r) = N_0^\top S_0^{-1} N_0 r + \sum_k N_k S_k^{-1} N_k^\top r$ with $S_0 = N_0 S N_0^\top$, and leave its application to optimization Schur complements as future work.

Their own PDE-constrained test problem does not exercise the difficulty: it cuts along breakpoints in time, so that “any set of two complicating variables are at most shared by one partition” — a one-dimensional interface with *no cross points at all*, where their ASd coincides with plain additive Schwarz. A two-dimensional tile partition is the regime the paper identifies as needing the second level, and peeling the cross points is the cheapest available form of it: they are placed in T , which is already built and already solved exactly, so the coarse correction costs $(k-1)^2$ extra columns and no new machinery.

7.3 Measured

Cameraman, exact Hessians, cross-point peel off versus on. The solution is unchanged in every row (identical PSNR to two decimals):

run	cross points	peel off	peel on
$N = 64$, 4 strips	0	129 it, 7.99 s	<i>bit-identical</i>
$N = 64$, 2×2 tiles	1	178 it, 11.4 s	170 it, 10.6 s
$N = 32$, 3×3 tiles	4	195 it, 6.31 s	113 it, 3.24 s
$N = 32$, 4×4 tiles	9	2251 it, 160 s	509 it, 36.8 s
$N = 64$, 4×4 tiles	9	did not converge	782 it, 173 s

Two features of this table are the argument. The benefit scales with the number of cross points, from nothing at one to a factor of four at nine; and strip partitions, which have no cross points, are *bit-identical* — the mechanism confirming itself rather than a general-purpose improvement that happens to help. The last row is the one that matters most: at $N = 64$ with 4×4 tiles the solver without the cross-point peel does not reach a solution at all (it is still grinding on dual infeasibility after ten minutes, having reached the final barrier level), and with it the run completes in under three minutes.

8 Preconditioned conjugate gradients on S_{ff}

8.1 The ASd preconditioner

A Krylov interface solve stands or falls on its preconditioner: with plain Jacobi ($M = \text{diag } |S_{\text{ff}}|$) every CG solve on a two-dimensional interface stalled in measurement, which is why Jacobi was removed from the implementation and only *ASd* (“additive Schur, assembled diagonal”, after equations (20)–(21) of the Lueg paper [13]) remains.

For each subdomain k , let R_k select the *kept* border positions that subdomain touches (a subset of N_k ; the rows of S_k belonging to peeled positions are dropped). Form the small dense block

$$M_k = R_k S_k R_k^{\top} \quad \text{with its diagonal replaced by} \quad (\text{diag } S)|_{R_k}, \quad (20)$$

using (14), and set

$$M^{-1} = \sum_{k=1}^K R_k^{\top} M_k^{-1} R_k. \quad (21)$$

The diagonal swap is the whole trick: S_k alone describes what subdomain k does to the shared border unknowns and badly underestimates their true self-coupling, to which every neighbouring subdomain also contributes; $\text{diag}(S)$ is the assembled truth, and the one quantity a distributed code can share with a single all-reduce. A singular local block M_k is simply skipped — a preconditioner may be incomplete; if what remains is useless, CG’s own breakdown guard (below) catches it.

8.2 The iteration

Algorithm 1 is textbook PCG [10] with three provisions that turn “CG assumes SPD” into a runtime check. The guard $\rho \leq 0$ certifies the preconditioner is not positive definite along the current residual; the guard $q^{\top} S_{\text{ff}} q \leq 0$ exhibits a direction of non-positive curvature. In exact arithmetic the latter is a one-directional *proof* of indefiniteness; in floating point at $\|A\| \sim 10^{18}$ a merely tiny $q^{\top} w$ can round non-positive, so the flag over-reports and is treated as *evidence*, *not certificate* (it is counted in the statistics but never acted on — runs where it fired a dozen

Algorithm 1 Preconditioned CG with breakdown guards and best-iterate memory

```
1:  $x \leftarrow 0$ ;  $r \leftarrow b$ ;  $z \leftarrow M^{-1}r$ ;  $\rho \leftarrow r^\top z$ ;  $q \leftarrow z$ 
2:  $x^* \leftarrow 0$ ;  $\rho^* \leftarrow \|b\|$  ▷ best iterate seen, and its residual norm
3: for  $t = 1, \dots, t_{\max}$  do
4:   if  $\|r\| < \rho^*$  then  $x^* \leftarrow x$ ,  $\rho^* \leftarrow \|r\|$ 
5:   end if
6:   if  $\|r\| \leq \tau \|b\|$  or no improvement for 40 iterations then break
7:   end if
8:   if  $\rho \leq 0$  then break ▷ preconditioner not SPD
9:   end if
10:   $w \leftarrow S_{\text{ff}} q$  ▷ via (13), embedded
11:  if  $q^\top w \leq 0$  then flag indefinite; break ▷ operator not SPD along  $q$ 
12:  end if
13:   $\alpha \leftarrow \rho / q^\top w$ ;  $x \leftarrow x + \alpha q$ ;  $r \leftarrow r - \alpha w$ 
14:   $z \leftarrow M^{-1}r$ ;  $\rho' \leftarrow r^\top z$ ;  $q \leftarrow z + (\rho' / \rho) q$ ;  $\rho \leftarrow \rho'$ 
15: end for
16: return  $x^*$  and its relative residual
```

times still predicted the inertia correctly on 78/79 factorizations). Finally the *best* iterate, not the last, is returned, so a stalled solve still hands back something usable.

8.3 Acceptance, and what happens on rejection

After the peel recombination (17)–(18) assembles the full Δy , the answer is judged by the residual of the *full* interface system — not the S_{ff} subproblem CG actually saw, because the peel algebra sits between the two:

$$\frac{\|r_y - S \Delta y\|}{\|r_y\|} < 10^{-2} \quad \text{and} \quad (\text{CG's own relative residual}) < 10^{-2}.$$

A rejected answer has nowhere to fall back to — S was never factorized — so it is handed back *anyway*, as the best iterate available, and the outer iterative refinement (Section 10) judges it against the true matrix. Reporting SINGULAR instead would be truthful but useless: IPOPT would answer with δ_w , which cannot repair a Krylov convergence failure, and the run dies (measured, in the historical no-inertia mode: internal error on three of four configurations). Only a solve that produced no iterate at all is reported as failed.

8.4 Relation to the iterative strategy of Lueg et al., §3.1

The interface solve of this solver is an implementation of the iterative strategy of [13], §3.1 — PCG on the Schur complement system, never forming or factorizing S , with a mandatory preconditioner, local step recovery by parallel subdomain back-solves (their eq. 11 = our (7)), local inertia read from the pivot signs of the W_k factorizations, and negative-curvature monitoring inside PCG — with two departures, one deliberate and one forced, and two points on which this implementation realizes what their paper lists as future extensions.

Deliberate departure: how S is applied. Their eq. (12) applies $S \cdot u$ matrix-free, at one W_k back-solve per partition per CG iteration, so that not even the local Schur blocks exist. Here

the S_k are formed once per factorization (Section 4) and each application is a dense $p_k \times p_k$ product. The operator and the one-reduction communication pattern are identical; only the local cost model differs, and the measurements of Section 4 decide it: the matrix-free application costs roughly ten times the dense product per iteration, and at the observed 10^2 – 10^3 CG iterations per factorization the formation cost is repaid twenty-fold. Two further facts tilt the trade. Their own preconditioners (eqs. 17 and 20–21) require the local Schur blocks regardless, and their cost model for formation — p_k full back-solves per partition — is exactly what the forward-only, static-reach route of Section 4 removes.

Forced departure: the peel. Their premise “ S is positive definite through inertia correction” is a structural fact of their formulation: the complicating variables are primal-only and appear only in linear linking constraints, so no dual ever reaches their border (and their corner block is zero, where our pure-permutation C is not). In the present formulation the border carries promoted corner duals — S is indefinite by construction — plus α and the cross points, so §3.1 is inapplicable to S itself. The peel of Section 6 is the adaptation that restores their premise: S_{ff} plays the role their S plays, and CG runs there.

Inertia checks. For the iterative mode their §3.5 offers two ways to check $\text{In}(S) = (p, 0, 0)$: monitoring $p^\top S p \geq 0$ inside PCG (“this is not a rigorous check,” their words), and a conservative per-block test $\text{In}(S_k) = (p_k, 0, 0)$. The first is precisely the pAp guard of Algorithm 1, retained here as guard and telemetry. The second is inapplicable in this setting: our S_k carry dual contributions and are indefinite by design. The prediction of Section 9 replaces both with an exact count.

Their future work, implemented here. Their implementation notes record that the absence of a filter line search and of iterative refinement forced PCG to a 10^{-9} relative residual, name both as “sensible extensions,” and flag the interplay of iterative tolerance with step quality as future research. This solver has both — it sits inside IPOPT’s filter line search, and Section 10 is the refinement — which is what lets the acceptance threshold sit at 10^{-2} ; the tolerance sweep reported in Section 11 is the experiment their future-work paragraph describes. One element of their §3.5 is genuinely not adopted: *localized* regularization (δ_H^k, δ_C^k adjusted only in the offending partition) is unavailable to a solver behind IPOPT’s linear-solver interface, where the regularization arrives already applied globally and the contract is limited to reporting SINGULAR or wrong inertia.

9 The predicted inertia

9.1 Haynsworth, a second time

The classical objection to a Krylov interface solve inside IPOPT is that the inertia (8) still needs $\text{In}(S)$, which classically comes only from a factorization of S — so S would be assembled and factorized anyway, the direct back-solve would then be nearly free, and CG would be strictly overhead. The resolution is to *derive* $\text{In}(S)$ from something already at hand. Theorem 1 applies to the peel split (15) exactly as it applied to the arrowhead:

$$\text{In}(S) = \text{In}(S_{\text{ff}}) + \text{In}(T), \quad T = S_{\text{PP}} - S_{\text{ff}}^\top S_{\text{ff}}^{-1} S_{\text{ff}}, \quad (22)$$

and T is *already built* — it is the peel cache of Section 6, of size $|P| \leq$ a few dozen, so $\text{In}(T)$ costs a dense symmetric eigendecomposition of a tiny matrix: nothing. The entire prediction is therefore the single assumption

$$\text{In}(S_{\text{ff}}) = (|f|, 0, 0), \quad \text{i.e. } S_{\text{ff}} \succ 0. \quad (23)$$

What makes (23) the right assumption to bet on is that it is *also exactly what CG requires*: the peel exists precisely to make S_{ff} definite. The scheme is self-consistent — the interface operator is usable exactly when the reported inertia is right — and the reported total is

$$n_-(A) = \sum_{k=1}^K n_-(W_k) + n_-(T). \quad (24)$$

Three implementation details keep (24) trustworthy. First, $\text{In}(T)$ is computed by an exact dense *symmetric eigendecomposition*, deliberately not by a dense LDL^T : Eigen’s dense LDLT is a pivoted Cholesky for semidefinite matrices, not Bunch–Kaufman, and its pivot signs are unreliable on indefinite input (the production solver measured 485–491 reported negatives where the truth was 512). Second, T is diagonally *equilibrated* before the eigendecomposition: $T \rightarrow DTD$ with $D = \text{diag}(|T_{ii}|^{-1/2})$ is a congruence, so $\text{In}(DTD) = \text{In}(T)$ by Sylvester’s law, while the eigenvalue scales become comparable — necessary because T mixes the α direction (barrier-scaled, up to $\sim 10^{20}$) with corner-dual directions ($\sim 10^{-9}$), and a zero test against the *global* eigenvalue scale mistakes honest tiny eigenvalues for noise (measured: at $N = 64$, 4×4 tiles it refused every factorization this way, and the run could not start; a referee LDL^T of the assembled S scored the equilibrated test’s predictions 199/200 correct). Third, an eigenvalue of the equilibrated matrix with $|\lambda| < 10^{-12} \max_i |\lambda_i|$ still means $\text{In}(T)$ is not well determined, and the prediction is refused.

9.2 Refusing to predict

If the peel cache cannot be built — a CG solve for a column of Z fails to converge, or T has an undeterminable eigenvalue — *no prediction is issued and the factorization is reported SINGULAR*. This is the safety property of the whole design: a prediction the solver cannot stand behind would corrupt IPOPT’s δ_w loop silently, whereas **SINGULAR** merely costs a re-factorization at a larger δ_w — a failure IPOPT knows how to cure.

9.3 Why $n_-(S)$ is predictable at all

Write $n_-(A) = m + \nu$, where m is the number of dual unknowns and ν the negative-curvature excess of the reduced Hessian — the quantity IPOPT drives to zero with δ_w . Each W_k is itself a saddle-point matrix, so $n_-(W_k) = m_k + \nu_k$ with m_k its interior duals. Every dual row is either owned by one subdomain or promoted to the border, so $\sum_k m_k = m - p_{\text{dual}}$, where p_{dual} is the number of promoted (peeled) border duals. Substituting into (8),

$$n_-(S) = p_{\text{dual}} + \nu - \sum_{k=1}^K \nu_k, \quad (25)$$

which explains the measured fact that $n_-(S)$ equals the promoted-dual count whenever the local and global curvature counts agree ($\nu = \sum_k \nu_k$, e.g. both zero at accepted steps). The $\text{In}(T)$ route (22) is used in practice because it needs no such assumption about ν : T sees whatever the corner duals actually contribute.

10 Iterative refinement and the outer solve

Every solve of the full system runs the arrowhead recursion (5)–(7) inside a few sweeps of iterative refinement measured against the *original input triplets*:

$$\text{repeat up to 3 times: } \rho = b - Ax \text{ (triplet matvec), } \quad x \leftarrow x + \mathcal{A}^{-1}\rho, \quad (26)$$

where $\mathcal{A}^{-1}$ denotes one arrowhead solve, stopping early at relative residual 10^{-11} and always returning the *best* iterate seen. This is not a luxury. Near-singular pivots — IPOPT’s own $\delta_c = 10^{-8}\mu^{1/4}$ on the dual directions, for instance — lose about ten digits through the Schur recursion (measured: relative residual ~ 0.1 unrefined against $\sim 10^{-12}$ refined). Because the residual in (26) is evaluated straight from the input triplets, refinement checks the *decomposition itself*, not merely the arithmetic inside it; it is also the safety net that absorbs the accepted-anyway CG iterates of Section 8.3. Returning one’s best answer at a nasty iterate and letting the globalization cope is exactly what a monolithic sparse solver does too. The solve reports failure upward only if the residual never once evaluated finite.

10.1 Life cycle

set_structure	once per sparsity pattern: number the unknowns, route every triplet to its block (W_k , B_k , or C), detect partition leaks (3), build the peel sets, run the symbolic analysis of every W_k .
factorize	every Newton step: (1) factorize all W_k (9), accumulate $\sum_k \text{In}(W_k)$ — any breakdown $\Rightarrow$ SINGULAR; (2) form all S_k by (11)–(12); (3) assemble C and $\text{diag}(S)$ (14); (4) build the peel cache (S_{fp} , S_{pp} , Z , T , $\text{In}(T)$) and the ASd blocks — failure $\Rightarrow$ refusal $\Rightarrow$ SINGULAR; report (24). Steps 1–2 are independent per subdomain.
solve	per right-hand side: refinement loop (26) around the arrowhead recursion; the interface step (6) is Algorithm 1 plus the peel recombination (17)–(18).

10.2 Constants

quantity	value	role
cg_tol	10^{-10}	CG target relative residual
cg_maxit	500	CG iteration cap
stall window	40	iterations without improvement before giving up
acceptance	10^{-2}	max relative residual (CG and full interface)
peel-column acceptance	10^{-2}	max relative residual for a Z column
refinement target	10^{-11}	early exit of (26)
refinement sweeps	3	maximum corrections
In(T) threshold	$10^{-12} \ \lambda\ _{\max}$	below: refuse to predict

11 What the measurements decided

The single configuration described above — CG-only interface, ASd-only preconditioning, predicted-only inertia — is the surviving point of a larger design space measured in this codebase (the production solver `dd_solver.hpp` retains the rest: MA57/MA97/MUMPS backends, a direct and a MINRES interface route, lagged Schur caching, a nested second level). The decisive measurements, all on the cameraman image with exact Hessians:

Correctness against a monolithic reference. Against IPOPT’s own MUMPS on identical instances, the decomposition changes the answer not at all: identical α^* to six digits and identical PSNR at every size/partition tested; the $\sim 15\%$ extra IPOPT iterations are the unpivoted LDL^T asking for regularization MA57 would not have needed (Section 3).

The preconditioner is everything. With ASd, CG carried every interface solve on strips and nearly all on tiles; with Jacobi on a 2×2 tile interface, CG carried *none* (0/50). Hence Jacobi’s removal.

The three inertia modes (historical; *exact* assembled and factorized S , *none* reported no inertia and used the curvature test of [4], *predicted* is Section 9):

run	exact	predicted	none
$N = 16$, 2 strips	50 it, 0.11 s	50 it, 0.09 s	71 it, 0.11 s
$N = 16$, 2×2 tiles	50 it, 0.10 s	80 it, 0.21 s	108 it, 0.24 s
$N = 32$, 3 strips	75 it, 1.49 s	80 it, 1.48 s	115 it, 1.83 s
$N = 32$, 3×3 tiles	75 it, 1.72 s	125 it, 4.51 s	1051 it, 63.3 s

All modes reach the same solution (PSNR identical to two decimals). Scored against a referee factorization of S , the prediction was correct on 78/79, 99/99, 111/117 and 132/133 factorizations respectively, and every mismatch was off by exactly one. On strips the prediction is *free* — same iterations and wall clock as exact inertia, with the one serial factorization gone — which is the result the design was after. What still costs on tiles is not wrong predictions but *refusals*: peel-cache columns whose CG solve does not converge. At $N = 64$ with 4×4 tiles the refusals become total (every factorization refused; the run cannot start), so the remaining obstacle of the approach is preconditioning the peel-column solves — a tractable, distributed-friendly problem — and no longer “we must factorize S to know its inertia.”

Formation of S_k . Forward-only + sparse-RHS (Sections 4–4.2) gave $1.65\text{--}1.8\times$ whole-run speedups over the naive route at $N = 32\text{--}64$; the row contraction (Section 4.3) a further $1.1\text{--}2.6\times$.

Validation gates. A standalone smoke test (`dd_simple_smoke.cpp`, pure Eigen, no IPOPT) checks the arrowhead solve against a dense LU and the predicted inertia (24) against a dense symmetric eigendecomposition, on synthetic arrowhead KKT matrices with and without the peel. `DDS_SCHUR_CHECK=1` referees every S_k at runtime; `DDS_DEBUG=1` prints partition and CG diagnostics.

References

- [1] A. Wächter and L. T. Biegler, *On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming*, Mathematical Programming 106 (2006) 25–57.
- [2] E. V. Haynsworth, *Determination of the inertia of a partitioned Hermitian matrix*, Linear Algebra and its Applications 1 (1968) 73–81.
- [3] R. J. Vanderbei, *Symmetric quasidefinite matrices*, SIAM Journal on Optimization 5 (1995) 100–113.
- [4] N.-Y. Chiang and V. M. Zavala, *An inertia-free filter line-search algorithm for large-scale nonlinear programming*, Computational Optimization and Applications 64 (2016) 327–354.
- [5] J. R. Gilbert and T. Peierls, *Sparse partial pivoting in time proportional to arithmetic operations*, SIAM Journal on Scientific and Statistical Computing 9 (1988) 862–874.
- [6] J. Mandel and R. Tezaur, *On the convergence of a dual-primal substructuring method*, Numerische Mathematik 88 (2001) 543–558.
- [7] A. Klawonn, O. B. Widlund, and M. Dryja, *Dual-primal FETI methods for three-dimensional elliptic problems with heterogeneous coefficients*, SIAM Journal on Numerical Analysis 40 (2002) 159–179.
- [8] A. Klawonn and O. Rheinbach, *Inexact FETI-DP methods*, International Journal for Numerical Methods in Engineering 69 (2007) 284–307.
- [9] X. Tu, *Three-level BDDC in three dimensions*, SIAM Journal on Scientific Computing 29 (2007) 1759–1780.
- [10] M. R. Hestenes and E. Stiefel, *Methods of conjugate gradients for solving linear systems*, Journal of Research of the National Bureau of Standards 49 (1952) 409–436.
- [11] C. Farhat, M. Lesoinne, P. LeTallec, K. Pierson, and D. Rixen, *FETI-DP: a dual-primal unified FETI method I*, International Journal for Numerical Methods in Engineering 50 (2001) 1523–1544.
- [12] C. R. Dohrmann, *A preconditioner for substructuring based on constrained energy minimization*, SIAM Journal on Scientific Computing 25 (2003) 246–258.
- [13] L. R. Lueg, M. L. Bynum, C. D. Laird, and L. T. Biegler, *Domain decomposition preconditioners for Schur complement systems arising in structured nonlinear optimization problems*, Optimization and Engineering 27 (2026) 555–585 (published online 2 September 2025), doi:10.1007/s11081-025-10020-1. The ASd preconditioner used here is their equation (21); the two-level correction discussed in Section 7 is their equation (22).